

Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

on laboratory work no. 1

Course: Network Programming

Topic: Concurrency in Practice

Done by:

st. gr. FAF-242, Bogdan Aramă

Checked by:

Fiștic Cristofor

Chișinău, 2026

Contents

1 Purpose of the work **2**

2 Source code **2**

3 Work environment **2**

4 Data and method of measurement **2**

5 Task 1: One file **3**

5.1 Task 1a: Counting with threads 3

5.2 Task 1b: Three ways to count 4

5.3 Task 1c: Producer and consumers 6

5.4 Answers to the questions of Task 1 7

6 Conclusion **8**

1 Purpose of the work

The aim of this laboratory work is to measure how the number of threads and the way the work is divided among them change the running time and the memory of a real workload: counting the negative, positive and zero values among 50 000 000 integers stored in a text file. From my own measurements I decide when this work is CPU-bound and when it is I/O-bound, I observe a race condition and the cost of fixing it, and I find where the parallel speed-up stops.

2 Source code

The program is written in Java and is kept in the Git repository

`https://github.com/bogdan-arama/pr-labs` (folder `lab1`)

It is one program, `lab1.Main`; every task and configuration is chosen by command-line parameters, for example `java -cp out lab1.Main 1a -threads 8`. The `README.md` in the folder explains how to build it (`javac -d out src/lab1/*.java`), how to generate the data and how to run every task. Two small Python scripts run all configurations (`scripts/run_benchmarks.py`) and draw the charts of this report (`scripts/plot.py`); all raw measurements are in `results/*.csv`.

3 Work environment

Item	Value
Computer	laptop, connected to mains power
CPU	AMD Ryzen 5 5600H (Zen 3), 3.3 GHz base, up to 4.2 GHz boost
Cores	6 physical cores, 12 logical processors (SMT, 2 threads per core)
RAM	16 GB DDR4-3200 (2 modules, 15.4 GB usable)
Disk	Intel SSD 670p 512 GB (`SSDPEKNU512GZ`), NVMe, PCIe 3.0 x4
Operating system	Windows 11 Pro, build 10.0.22624, 64-bit
Language and runtime	Java 17, Eclipse Temurin OpenJDK 17.0.18+8, HotSpot 64-Bit Server VM
Helper scripts	Python 3.14.2, matplotlib 3.10.8

Table 1: Hardware and software used for all measurements

4 Data and method of measurement

Data. `java -cp out lab1.Main gen` writes `data/numbers.txt`: 50 000 000 integers drawn uniformly from $[-1\,000\,000, 1\,000\,000]$ with `SplittableRandom` (seed 42), one per line, 369 445 634 bytes. While writing, the generator also counts the signs and saves them in `numbers.txt.expected`.

Correctness check. The expected result is **25 001 107 negative, 24 998 868 positive and 25 zero** values, which add up to 50 000 000. After every run the program compares its totals with this file and prints `correct=true` or `false`; the benchmark script stops if any correct version gives wrong totals. Every run of Tasks 1a, 1b (versions 2 and 3) and 1c produced exactly these numbers.

Time. `System.nanoTime()` is read just before the threads are created and just after the last one has been joined. JVM start-up is not included, and in Task 1b loading the array is not included either, as the task requires.

Peak memory. At the end of every run the program asks the operating system for the peak working set of its own process (`PeakWorkingSet64` on Windows, `VmHWM` on Linux). This is the largest amount of physical memory the whole process (JVM, heap, thread stacks, native buffers) used during the run. To make it belong to one configuration only, every configuration is run in a fresh JVM.

Repetitions. Tasks 1a and 1c were run 5 times per thread count, Task 1b 3 times; the charts show the median and a shaded band from the fastest to the slowest run. Repetitions were interleaved (all thread counts once, then all again), so slow changes of the machine (temperature, background programs) spread over all configurations. Besides the thread counts required by the task (1, 2, 4, 8, 16, 32, 64) I also measured 6 and 12, the numbers of physical cores and logical processors. In the charts these two are marked with dotted vertical lines. Speed-up is the median time on 1 thread divided by the median time on N threads.

File cache. The 369 MB file easily fits in the 16 GB of RAM, so after the first read Windows keeps it in its file cache. Before Tasks 1a and 1c the benchmark makes one unmeasured run to bring the file into the cache, so all measured runs read it from memory, not from the SSD.

5 Task 1: One file

5.1 Task 1a: Counting with threads

Implementation (`Task1a.java`). With N threads the file of S bytes is divided into N byte ranges $[\, S \cdot i/N,\ S \cdot (i+1)/N \,)$. Every thread opens its own `RandomAccessFile`, seeks to the start of its range and reads it in 1 MiB blocks into its own buffer, parsing the bytes as they come (`LineParser`: digits are accumulated, a `-` sets the sign, `\n` ends the number) and counting into its own `Counts` object. The main thread joins the threads and adds their counts. No thread reads the whole file.

A byte range usually starts and ends in the middle of a line, so one rule decides who owns each line: *a line belongs to the range in which its first byte lies.* Two consequences follow:

- a thread first looks at the byte just before its range; if it is not `\n`, the range starts inside a line that the previous thread owns, so the thread skips bytes up to and including the next `\n`;
- a thread keeps reading after the end of its range until it finishes the last line it has started, and it stops as soon as the next line would start at or after its end.

Because every line has exactly one first byte, and that byte is in exactly one range, every line is counted by exactly one thread. I checked the rule on a small file with every thread count from 1 to 64 (more threads than lines included), with `\r\n` line ends and without a final newline, and on the real file: every run gave exactly 50 000 000 numbers and the expected totals.

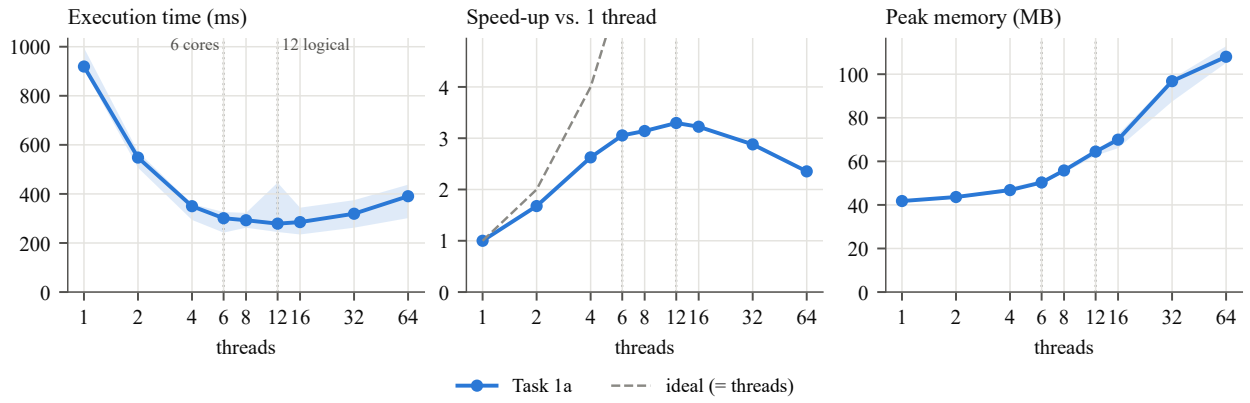


Figure 1: Task 1a: time, speed-up and peak memory against the number of threads (median of 5 runs; the band shows the fastest and slowest run; dotted lines: 6 physical cores, 12 logical processors)

Threads	Median time (ms)	Min–max (ms)	Speed-up	Peak memory (MB)
1	919	908–996	1.00	42
2	548	507–572	1.68	44
4	350	294–359	2.63	47
6	301	242–326	3.05	50
8	293	262–323	3.14	56
12	279	245–446	3.30	64
16	285	235–344	3.22	70
32	319	262–375	2.88	97
64	391	301–438	2.35	108

Table 2: Task 1a results

5.2 Task 1b: Three ways to count

Implementation (`Task1b.java`). The whole file is first loaded into an `int[]` of 50 000 000 elements (not measured). The array is divided into equal contiguous parts, one per thread, and the program is run with `-mode unsync|lock|local`. By default it uses as many threads as there are logical processors, `Runtime.availableProcessors()` = 12 on my laptop. The three versions differ only in where a thread adds one to a counter:

```
// 1: shared counters (plain fields of one shared object), no synchronisation
if (v < 0) shared.negative++; else if (v > 0) shared.positive++; else shared.zero++;

// 2: the same shared counters, every increment under a ReentrantLock
lock.lock();
try { if (v < 0) shared.negative++; else if (v > 0) shared.positive++; else shared.zero
    ++; }
finally { lock.unlock(); }

// 3: local variables of the thread; written to its own Counts once, summed after join()
if (v < 0) negative++; else if (v > 0) positive++; else zero++;
```

Table 3 shows every run with 12 threads (version 1 three times, as required, and the other versions three times as well). Figure 2 and Table 4 show the three versions for all thread counts.

Version	Run	Time (ms)	Negative	Positive	Zero	Total	Correct
1: shared, no sync	1	330.1	3,369,126	3,360,083	25	6,729,234	**no**
1: shared, no sync	2	348.5	3,954,164	3,943,448	25	7,897,637	**no**
1: shared, no sync	3	331.2	3,402,233	3,391,194	25	6,793,452	**no**
2: shared + lock	1	794.6	25,001,107	24,998,868	25	50,000,000	yes
2: shared + lock	2	767.5	25,001,107	24,998,868	25	50,000,000	yes
2: shared + lock	3	736.1	25,001,107	24,998,868	25	50,000,000	yes
3: per-thread counters	1	72.7	25,001,107	24,998,868	25	50,000,000	yes
3: per-thread counters	2	83.3	25,001,107	24,998,868	25	50,000,000	yes
3: per-thread counters	3	74.8	25,001,107	24,998,868	25	50,000,000	yes

Table 3: Task 1b with 12 threads (= logical processors): time and totals of every run. The correct totals are 25 001 107 / 24 998 868 / 25 / 50 000 000

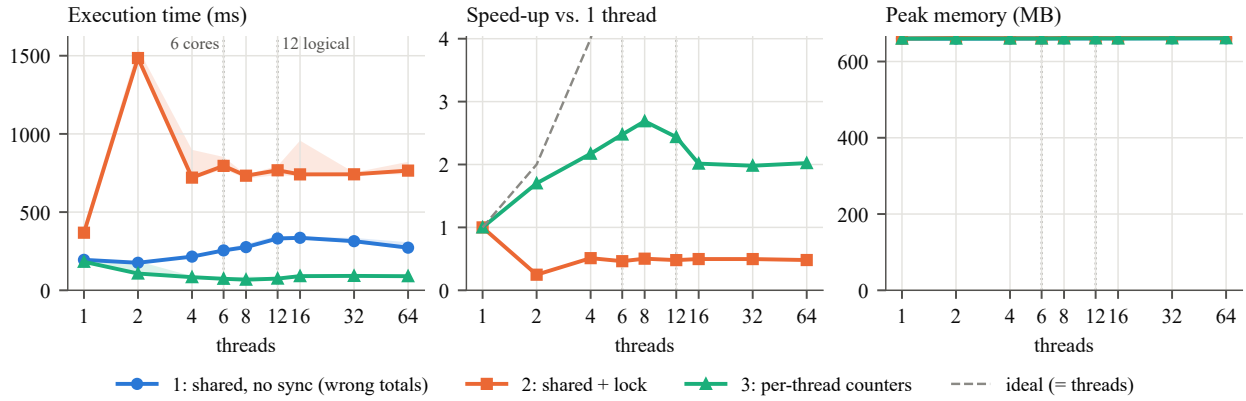


Figure 2: Task 1b: the three versions against the number of threads (median of 3 runs). Version 1 has no speed-up curve because its results are wrong. The peak-memory lines lie on top of each other

Threads	1: no sync (ms)	2: lock (ms)	3: per-thread (ms)	1: median total
1	195	368	182	50.0M
2	176	1485	107	32.3M
4	215	721	84	21.3M
6	255	796	74	14.9M
8	276	733	68	12.2M
12	331	767	75	6.8M
16	336	741	90	7.2M
32	314	742	92	6.7M
64	273	765	90	6.6M

Table 4: Task 1b: median time of each version, and the median total counted by version 1 (it should be 50.0M)

The peak memory is about 660 MB for every version and thread count. It comes from loading the array: the array grows by doubling while the file is read and is then trimmed to exactly 50 000 000 elements, so for a moment a 256 MB and a 200 MB array exist at the same time. The counters themselves take a few bytes, so the memory does not depend on the version or the number of threads.

5.3 Task 1c: Producer and consumers

Implementation (`Task1c.java`). The reader thread reads `numbers.txt` line by line (`readLine()` of a `BufferedReader`) and collects the lines into `String[]` batches of 10 000. It puts each batch into an `ArrayBlockingQueue` with capacity 100 batches, a blocking queue from the Java library. `put()` blocks the reader while the queue is full, and `take()` blocks a worker while the queue is empty. Java's `BlockingQueue` has no `close()`, so the reader closes the queue another way. After the last batch it puts one empty marker array (`END`) for every worker. The queue is FIFO, so a worker sees `END` only after all real batches are taken, and then it stops. Every worker parses the lines of its batches and counts into its own counters, which are added after `join()`. The program also counts how often the reader found the queue full and how often a worker found it empty, and the largest number of batches that were waiting in the queue.

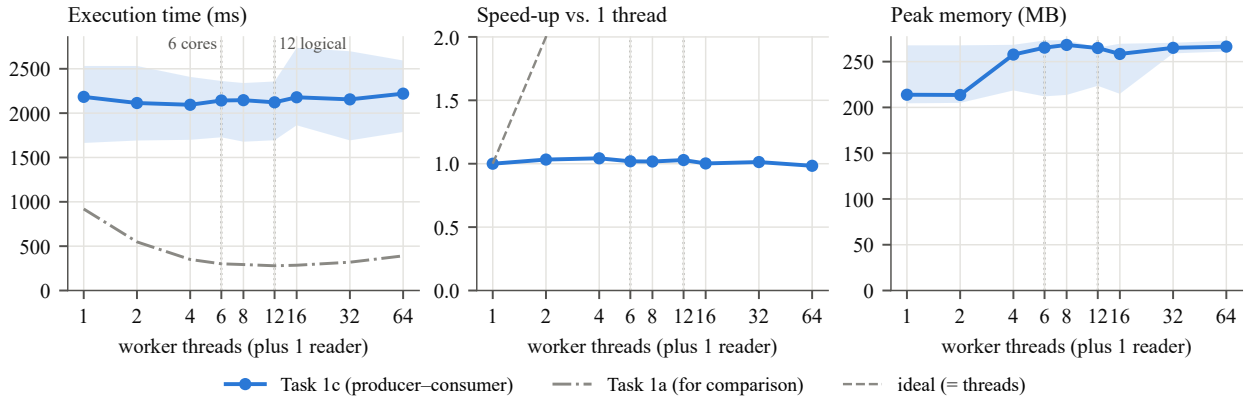


Figure 3: Task 1c: time, speed-up and peak memory against the number of workers (median of 5 runs; the band shows the fastest and slowest run). The dash-dotted line is Task 1a

Threads	Median time (ms)	Min–max (ms)	Speed-up	Peak memory (MB)
1	2183	1664–2532	1.00	214
2	2114	1691–2532	1.03	214
4	2094	1699–2408	1.04	258
6	2143	1727–2362	1.02	265
8	2146	1678–2340	1.02	268
12	2122	1694–2357	1.03	265
16	2179	1863–2736	1.00	258
32	2155	1693–2698	1.01	265
64	2220	1788–2594	0.98	266

Table 5: Task 1c results

Workers	Worker waits (empty queue)	Reader waits (full queue)	Max batches in queue
1	4975	0	48
2	4999	0	6
4	5001	0	6
6	5003	0	4
8	5006	0	5
12	5011	0	1
16	5012	0	2
32	5029	0	2
64	5061	0	5

Table 6: Task 1c: how the queue was used (the file gives 5 000 batches; waits are medians of 5 runs, the queue length is the largest over the runs)

To answer question 6 by measurement and not only in words, I also ran Task 1c with batches of a single line (`-batch 1`) and with a queue without a size limit (`-capacity 0`, which uses a `LinkedBlockingQueue`), with 1, 4 and 12 workers (Table 7, one run each; the rows with batch 10 000 and limit 100 are the medians from Table 5).

Batch (lines)	Queue limit	Workers	Time (ms)	Peak memory (MB)	Max batches in queue	Worker waits
10,000	100	1	2183	214	48	4975
1	100	1	13377	207	100	1428281
10,000	no limit	1	2191	358	65	4475
10,000	100	4	2094	258	6	5001
1	100	4	19320	205	100	963760
10,000	no limit	4	2143	258	1	5003
10,000	100	12	2122	265	1	5011
1	100	12	42159	205	100	932549
10,000	no limit	12	2094	269	2	5011

Table 7: Task 1c with other batch sizes and queue limits

5.4 Answers to the questions of Task 1

1. At which number of threads does the speed-up stop growing? Relate it to your physical cores and logical processors.

The speed-up grows fast up to 6 threads, my 6 physical cores (3.05×), then only a little up to 12, my 12 logical processors (3.30×, the best). Two logical processors of one core share its execution units, so the second one adds only 8 %. Above 12 the time grows again.

2. Why do 32 and 64 threads not help? Are they slower, and why?

They are slower: 319 ms and 391 ms against 279 ms with 12 threads. Only 12 threads can run at the same time, so the operating system has to switch between the others, and every switch, thread start and extra read buffer only adds time.

3. Does the peak memory depend on the number of threads? Why?

A little in Task 1a: 42 MB with 1 thread and 108 MB with 64, about 1 MB per thread, because every thread has its own 1 MiB read buffer and stack. In Task 1b (about 660 MB, the loaded array)

and Task 1c (214–268 MB, the queue and the strings) it does not depend on the threads.

4. Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

With 12 threads: per-thread counters about 75 ms, no synchronisation about 330 ms, lock about 770 ms. Only the lock and the per-thread versions are correct.

- *No synchronisation*: wrong, 6.7–7.9 million instead of 50 million. `counter++` is read, add, write; two threads read the same value and one increment is lost.
- *Lock*: correct, but only one thread at a time can count, so it is slower than one thread alone (368 ms).
- *Per-thread counters*: correct and fastest, because the threads share nothing while counting and the results are added once after `join()`.

5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is faster: 279 ms against about 2 100 ms. In Task 1c only one reader reads the file and creates the strings, and the workers wait for it (they found the queue empty for almost every batch). So Task 1c does not get faster after 1 worker.

6. What would happen with batches of a single line? And with a queue that has no size limit?

With batches of one line every line needs its own queue operation and lock, so it was 6–20 times slower (13 s with 1 worker, 42 s with 12). A queue with no limit gave the same time here, because the reader is the slow one; but if the workers were slower, the whole file could pile up in memory (about 2.5 GB). The limit of 100 batches makes the reader wait and keeps the memory small.

6 Conclusion

On my laptop (6 cores, 12 logical processors) all the work I measured, counting 50 million numbers, was **CPU-bound**. I tell this by two checks: the time falls when threads are added (as long as there are free cores), and the time is close to the time of the pure computation, with no part spent waiting for a device.

- *Task 1a*: the time fell from 919 ms on 1 thread to 301 ms at 6 threads and 279 ms at 12. If the work were waiting for the disk, more threads would not have made it three times faster. Beyond the 12 logical processors it got worse, because there was no more processor to give.
- *Task 1b*: the data is already in memory, so there is no I/O at all. The per-thread version went from 182 ms on 1 thread to 68 ms at 8 threads.
- *Task 1c* is limited by the processor too, but by only *one* core. The single reader that reads the lines and creates the strings is the bottleneck: the workers find the queue empty for almost every batch, and 1 to 64 workers all take about 2.1 s.

None of these tasks was I/O-bound on my machine. The 369 MB file fits easily in the 16 GB of RAM, so Windows keeps it in its file cache, and "reading" it is only copying memory. Work

becomes I/O-bound when the data has to come from a device much slower than the processor, which these tasks did not do.

Synchronisation has a price that depends on how often it happens. A lock taken for every number made Task 1b ten times slower than per-thread counters (about 770 ms vs. 75 ms) and never faster than one thread. Batches of one line made Task 1c 6–20 times slower than batches of 10 000. Shared counters without synchronisation were both wrong (84–87 % of the increments lost) and over 4 times slower than per-thread counters (about 330 ms). The fastest correct design was to share nothing while working and combine the results once at the end.